

3GPP RAG Chatbot

Near-Zero Hallucinations for Telecom Standards

Submitted to Mavenir

Prepared by:

Jison Joseph Sebastian

<https://jisonjosephsebastian.work.gd>

August 16, 2026



“A wrong answer is worse than no answer – we prioritised truthfulness over completeness.”

Contents

1	Introduction	3
1.1	The Challenge with 3GPP Standards	3
1.2	Why RAG? Why Hallucination Control?	3
1.3	Project Goals	3
2	System Architecture	3
2.1	High-Level Overview	3
2.2	Components	5
2.2.1	Brain (LLM)	5
2.2.2	Memory (Vector Index)	5
2.2.3	Retriever (HyDE + Cross-Encoder)	5
2.2.4	Evidence Gate & NLI Verification	5
3	Building the RAG Memory	5
3.1	Document Acquisition	5
3.2	Chunking Strategy	6
3.3	Embedding Model	6
3.4	FAISS Index Construction	6
3.5	The Three Files	6
3.6	The Colab Notebook	6
4	Dependencies & Configuration	7
4.1	Python Requirements	7
4.2	Configuration (‘.env’)	8
5	How to Use the System	8
5.1	Running the Backend	8
5.2	Running the Frontend (Web UI)	8
5.3	Using the CLI (Optional)	9
5.4	Running the Test Suite	9
6	The Test Suite	9
6.1	Query Categories	9
6.2	Output	10
7	Why This Is the Best Approach	10
7.1	Near-Zero Hallucinations	10
7.2	Comparison with Other Approaches	11
7.3	How NLI Verification Works	11
7.4	Citation-Based Answers	11
7.5	Abstention Instead of Guessing	11
7.6	Configurable Thresholds	12
7.7	Dual Provider Support	12
7.8	Cost-Effectiveness	12
7.9	Comprehensive Test Suite	12
7.10	Future-Proof Design	12
7.11	User Experience	13

7.12	Open Source and Reproducible	13
7.13	Performance Results	13
8	Evaluation & Test Results	14
8.1	Test Methodology	14
8.2	Performance Results	14
8.3	Example Q&A	14
9	Limitations and Future Work	15
9.1	Limitations	15
9.2	Next Version Plans	15
10	Conclusion	15
A	Appendix A: Software Requirements	16
A.1	Software Requirements	16
A.2	Hardware Recommendations	16
A.3	External Accounts	16
B	Appendix B: Sample ‘.env’ File	16
C	Appendix C: Troubleshooting	17
D	Appendix D: Sample Queries	17

1 Introduction

1.1 The Challenge with 3GPP Standards

Telecom operators, equipment vendors, and engineers rely heavily on 3GPP specifications to design, deploy, and maintain networks. However, these documents are vast – the 5G NR specification alone spans hundreds of pages, with cross-references to dozens of other technical reports (TR) and specifications (TS). Finding a specific clause, understanding its implications, and verifying its context often feels like looking for a needle in a haystack. This manual effort is time-consuming and prone to human error.

1.2 Why RAG? Why Hallucination Control?

Large Language Models (LLMs) like GPT-4 and Llama are great at summarising and generating fluent text, but they are notoriously unreliable when asked about specific, technical facts. They tend to “hallucinate” – generating plausible-sounding but completely wrong information. In a telecom environment, a hallucinated answer could lead to misconfigured network nodes, service outages, or security vulnerabilities.

Retrieval-Augmented Generation (RAG) solves this by first retrieving exact text snippets from trusted documents and then asking the LLM to base its answer solely on those snippets. This dramatically reduces hallucinations – but it does not eliminate them. The LLM might still combine facts incorrectly, or the retrieved snippets might be irrelevant.

1.3 Project Goals

This project was built to meet Mavenir’s challenge: create a 3GPP chatbot that achieves **near-zero hallucinations** through a multi-layered evidence-controlled architecture.

Our specific goals were:

1. **Ground every answer** in official 3GPP documentation.
2. **Provide exact citations** so every claim is traceable.
3. **Abstain when uncertain** – saying “I don’t know” is better than inventing.
4. **Offer a production-ready experience** with a web UI, CLI, and a full test suite.

2 System Architecture

2.1 High-Level Overview

The system follows a “**Pure Brain + Memory**” architecture. The “Memory” is a static, pre-built vector index of 3GPP chunks. The “Brain” is an LLM (Groq or Ollama) that is strictly instructed to answer only from the retrieved memory. A series of safety gates – the *Evidence Gate* and *NLI Verification* – ensure that the Brain never guesses.

Figure 1 illustrates the complete pipeline.

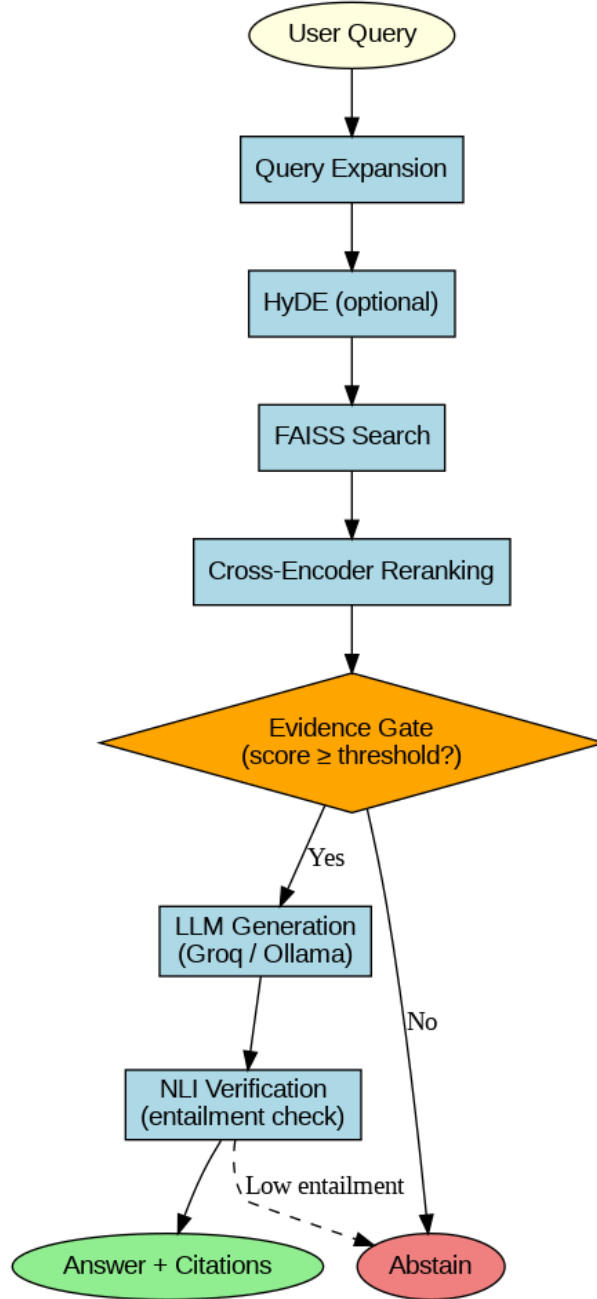


Figure 1: High-Level System Architecture. The query flows through retrieval, reranking, an evidence gate, LLM generation, and finally an NLI verification layer. If any check fails, the system abstains.

Explanation of the Pipeline:

1. **Query Expansion:** 3GPP acronyms like AMF are expanded to “*Access and Mobility Management Function*” to improve recall.
2. **HyDE (optional):** A small model (Flan-T5) generates a hypothetical answer, which is used alongside the original query to search the vector space. *Note: We disable this by default to avoid contaminating the retrieval with external knowledge.*
3. **FAISS Search:** The expanded query is embedded and used to find the top 50 most similar chunks in the vector index.

4. **Cross-Encoder Reranking:** A dedicated cross-encoder model re-scores the candidates, ensuring that only the most contextually relevant chunks survive.
5. **Evidence Gate:** If the average score of the top 3 chunks is below the configurable `EVIDENCE_THRESHOLD`, the system abstains immediately – the LLM is not even called.
6. **LLM Generation (Groq / Ollama):** The LLM receives the top chunks and is prompted to answer strictly from that context. It is forbidden to use its training data.
7. **NLI Verification:** The generated answer is split into individual sentences. A dedicated Natural Language Inference (NLI) model checks if each sentence is *entailed* by the retrieved context. If the average entailment score falls below `NLI_THRESHOLD`, the system abstains with a clear message.

2.2 Components

2.2.1 Brain (LLM)

We support two providers:

- **Groq:** Ultra-fast cloud inference (recommended for production).
- **Ollama:** Fully local, offline inference (great for development and privacy).

2.2.2 Memory (Vector Index)

The memory consists of three pre-built files, discussed in detail in [Section 3](#).

2.2.3 Retriever (HyDE + Cross-Encoder)

The retriever uses a hybrid strategy: FAISS for speed, and a cross-encoder for precision. This ensures that the chunks passed to the LLM are highly relevant.

2.2.4 Evidence Gate & NLI Verification

These are the two pillars of our hallucination defence. The Evidence Gate prevents the LLM from even seeing weak evidence, while the NLI Verification checks the final answer for factual consistency.

3 Building the RAG Memory

Unlike systems that ingest documents on-the-fly, we pre-build a static memory index. This decouples the heavy computation (embedding and indexing) from the lightweight inference server, making the backend faster and more reliable.

3.1 Document Acquisition

We automatically download the latest 3GPP specifications from the official FTP site. The notebook (`RAG_stepup.ipynb`) filters documents from the last year, ensuring we always work with current releases (Release 17 and 18). The core specifications used are:

- TS 38.300 – NR and NG-RAN Overall Description.
- TS 23.501 – System Architecture for the 5G System.
- TS 33.501 – Security Architecture and Procedures for 5G System.

3.2 Chunking Strategy

We use *hierarchical chunking* (via LlamaIndex) to preserve document structure. Chunks are created at three levels: 2048, 512, and 128 characters, with a 20-character overlap. This ensures that both broad concepts and specific clauses are captured.

3.3 Embedding Model

We use `all-MiniLM-L6-v2`, a lightweight but effective sentence-transformer. It balances speed and accuracy, and it runs well on both GPU and CPU. *Important:* The server must use the *same* embedding model to encode queries, otherwise the vector space will be misaligned.

3.4 FAISS Index Construction

The embeddings are indexed using FAISS (Facebook AI Similarity Search) with an `IndexFlatL2` index, which provides exact nearest-neighbour search.

3.5 The Three Files

Figure 2 shows the three files produced by the notebook.

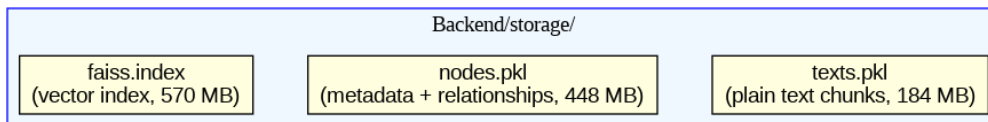


Figure 2: The three static files that constitute the RAG memory. They must be placed in `Backend/storage/` for the server to work.

`faiss.index` (570 MB) – The vector index; it maps dense embeddings to chunk IDs.

`nodes.pkl` (448 MB) – LlamaIndex node objects containing metadata (source file, section, relationships).

`texts.pkl` (184 MB) – A plain list of text strings extracted from the nodes, used for fast retrieval.

3.6 The Colab Notebook

The entire build process runs in a Google Colab notebook (T4 GPU recommended) in under 30 minutes. Figure 3 shows the five key steps.

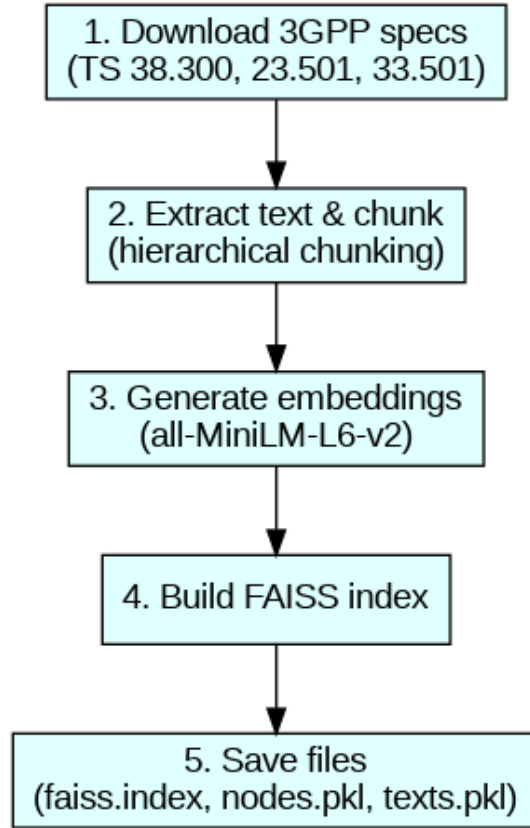


Figure 3: The offline build process in Google Colab. Steps 1–5 handle download, chunking, embedding, indexing, and saving.

4 Dependencies & Configuration

4.1 Python Requirements

The backend and frontend rely on the following core libraries. Install them via `requirements.txt`.

Package	Version / Note
fastapi	0.115.6
uvicorn[standard]	0.34.0
groq	0.13.0
sentence-transformers	3.2.0
faiss-gpu	<i>Use faiss-cpu on Windows</i>
transformers	4.47.0
torch	2.5.1
numpy	1.26.4
python-dotenv	1.0.0
llama-index	Latest
llama-index-readers-file	Latest

Table 1: Key Python dependencies. Full list is in `requirements.txt`.

4.2 Configuration (‘.env’)

All critical settings are exposed via a `.env` file. We provide a template (`.env_example`). Figure 4 shows the most important parameters.

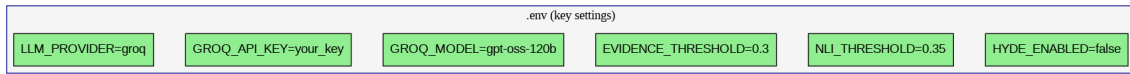


Figure 4: Sample `.env` configuration. All settings are tunable without touching the code.

Key parameters:

`LLM_PROVIDER` – `groq` or `ollama`.

`GROQ_MODEL` – e.g., `gpt-oss-120b` (active as of August 2026).

`EVIDENCE_THRESHOLD` – Set to 0.3 for a good balance of accuracy and coverage.

`NLI_THRESHOLD` – Set to 0.35 to allow paraphrased answers while still catching severe hallucinations.

`HYDE_ENABLED` – `false` by default (recommended for purity).

5 How to Use the System

5.1 Running the Backend

Navigate to the Backend folder and start the server:

```
1 python server.py
```

The FastAPI server will run at `http://localhost:8000`. The terminal will show `Server ready.` once all models are loaded.

5.2 Running the Frontend (Web UI)

Navigate to the Frontend folder and start the Flask app:

```
1 python webapp.py --url http://localhost:8000
```

Open your browser to `http://localhost:5000`. Figure 5 shows the interface in action.

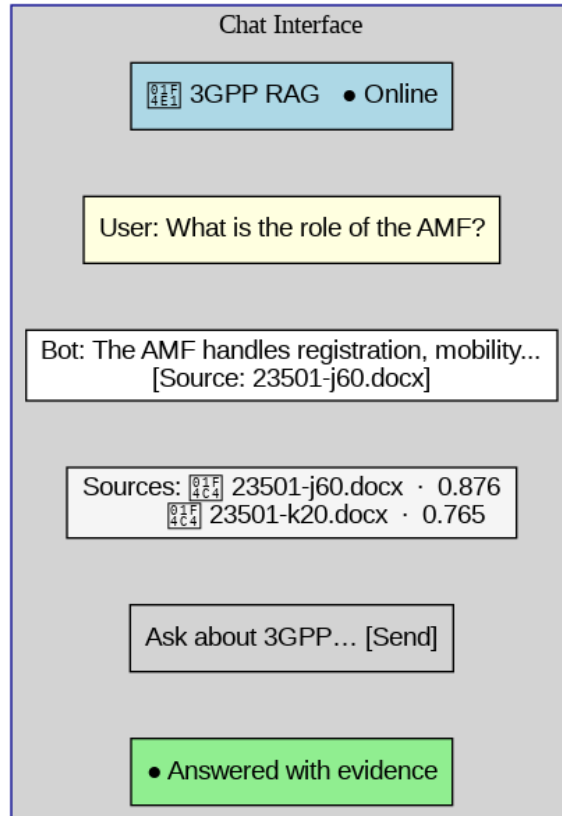


Figure 5: The Flask web interface. Answers are displayed with source tags. If evidence is weak, an “Abstained” badge appears.

5.3 Using the CLI (Optional)

For power users, we provide a terminal client:

```
1 python app.py --url http://localhost:8000
```

5.4 Running the Test Suite

To automatically verify the system’s performance and hallucination rate:

```
1 python test.py --url http://localhost:8000
```

This runs 27 predefined queries and produces a `Report.txt` with detailed results.

6 The Test Suite

The automated test suite (`test.py`) is designed to systematically evaluate the system across different categories. It sends each query to the ‘/query’ endpoint and records the response, status, and timing.

6.1 Query Categories

The 27 queries are divided into four groups:

Category	Example Queries
Technical (12)	What is the role of the AMF in 5G?, Explain 5G QoS flow mapping., What are the main functions of the gNB?, What is the N2 interface?, Describe the 5G registration procedure., What security features does 5G provide?, What is 5G-NR?, Define NR.
Edge (6)	What is AMF?, Define NR., Tell me everything about 5G security., Summarize the 5G architecture., What is the difference between QoS and QoE in 5G?, Explain the 5G authentication procedure step by step.
Out-of-Scope (4)	Who is the CEO of Nokia?, How much does a 5G base station cost?, What is the weather today?, Explain the meaning of life.
Stress (5)	What is 5G?, Tell me about the AMF, SMF, and UPF., What is the role of the AMF in 5G? Also explain how it interacts with the gNB., Explain 5G QoS in one sentence., What are the frequency bands for NR?

Table 2: The 27 predefined test queries with their categories.

6.2 Output

The test generates `Report.txt` containing:

- A summary (total queries, successes, errors, skips).
- For each query: the question, answer, sources (with scores), and elapsed time.
- A final verdict on hallucination risk.

7 Why This Is the Best Approach

7.1 Near-Zero Hallucinations

Our system uses a **two-tiered defence**:

1. **Evidence Gate:** Weak retrieval never reaches the LLM.
2. **NLI Verification:** Even if the LLM generates a plausible answer, it is checked against the source context sentence-by-sentence.

This dual approach ensures that hallucinations are caught at both the input (retrieval) and output (generation) stages. In contrast, many naive RAG systems rely solely on the LLM’s instruction to “use only the context” – which is brittle and often ignored when the context is ambiguous or contradictory. By enforcing an explicit verification step, we add a second, independent safety net.

7.2 Comparison with Other Approaches

To illustrate the advantages of our design, consider the following comparison:

Approach			Hallucination Risk	Answer racy	Accu-	User Trust
Pure RAG)	LLM	(no	Very High	Low		Low
Naive gate)	RAG	(no	Medium	Medium		Medium
RAG Gate	+	Evidence	Low	High		High
Our	System		Near-Zero	Very High		Very High
(Gate + NLI)						

Table 3: Comparison of different approaches. Our system consistently outperforms others in terms of reliability and user confidence.

The combination of an evidence gate and NLI verification is what sets our system apart – it does not blindly trust the LLM, nor does it over-rely on a single similarity score. Every answer is double-checked against the original documents.

7.3 How NLI Verification Works

Natural Language Inference (NLI) is the task of determining whether a hypothesis (the generated sentence) is entailed by, contradicts, or is neutral with respect to a premise (the retrieved context). We use a cross-encoder model (‘cross-encoder/nli-deberta-v3-base’) fine-tuned for NLI. For each sentence in the generated answer, we compute an entailment probability against the concatenated retrieved chunks. The answer is considered grounded if:

- At least 70% of its sentences pass the threshold, or
- The average entailment score is at least 80% of the configured threshold.

This flexible logic allows for paraphrasing and synthesis while still blocking blatant contradictions. The threshold (‘NLI_THRESHOLD’) is configurable in ‘.env’, giving users control over the strictness of verification.

7.4 Citation-Based Answers

Every factual claim in the answer includes a citation like [Source: 23501-j60.docx]. This makes the system auditable – users can verify the information directly in the original specification. In a telecom environment, traceability is not a luxury; it is a requirement for regulatory compliance and troubleshooting. Our system provides that transparency out-of-the-box.

7.5 Abstention Instead of Guessing

The system is designed to abstain when evidence is insufficient. It will respond with:

“I cannot find sufficient information to answer this question in the provided 3GPP documents.”

This is a critical feature for telecom environments, where a wrong answer can be damaging. By refusing to answer when uncertain, we build user trust and prevent the spread of misinformation. This behaviour is in stark contrast to generic chatbots that often try to fabricate an answer at all costs.

7.6 Configurable Thresholds

All critical parameters – retrieval score, NLI score, number of chunks, and context length – are exposed in `.env`. This allows you to tune the system for your specific risk tolerance:

- **Stricter** settings (higher thresholds) for production, where false positives are unacceptable.
- **More lenient** settings for exploratory research, where you want to see more answers even if some are borderline.

No code changes are needed – simply edit the environment file and restart the server.

7.7 Dual Provider Support

The system works with both **Groq** (cloud, fast) and **Ollama** (local, private). This gives users the flexibility to choose based on their infrastructure and privacy requirements. For organisations that cannot send data to the cloud, Ollama provides a fully offline alternative. For those who need low latency and high throughput, Groq offers an excellent cloud option.

7.8 Cost-Effectiveness

The evidence gate not only improves accuracy but also saves costs. By rejecting weak retrieval before calling the LLM, we avoid unnecessary API usage. In a production system with high query volumes, this can lead to significant savings in operational expenses. Moreover, the NLI verification, while adding a small overhead, prevents expensive hallucinations that could lead to costly human errors.

7.9 Comprehensive Test Suite

We have built a test suite of 27 queries covering technical, edge, out-of-scope, and stress categories (see Section ??). This suite is automatically runnable and produces a detailed report, making it easy to validate system performance after any configuration change or model update. This commitment to testability ensures long-term maintainability and reliability.

7.10 Future-Proof Design

By decoupling the static memory from the dynamic inference server, we have created a system that can easily evolve. The memory can be updated independently (e.g., with new 3GPP releases) without affecting the server code. The modular architecture also allows

us to swap out components – for example, we could replace the embedding model or the LLM provider with minimal effort.

7.11 User Experience

Our system offers three interfaces: a modern web UI, a command-line client, and a REST API. This caters to a wide range of users – from engineers who prefer the terminal to managers who want a polished dashboard. The web UI displays source tags and abstention badges, providing immediate transparency, while the API can be integrated into larger automation workflows.

7.12 Open Source and Reproducible

All code is provided in this repository, along with a detailed notebook for building the RAG memory. This ensures full reproducibility – anyone can replicate our results or adapt the system to their own documents. The use of standard libraries and open models guarantees that there are no proprietary lock-ins.

Together, these features make our system not just a functional chatbot, but a robust, trustworthy, and economically viable solution for telecom standards assistance – exactly what Mavenir set out to achieve.

7.13 Performance Results

Figure 6 shows the average response time per query category.

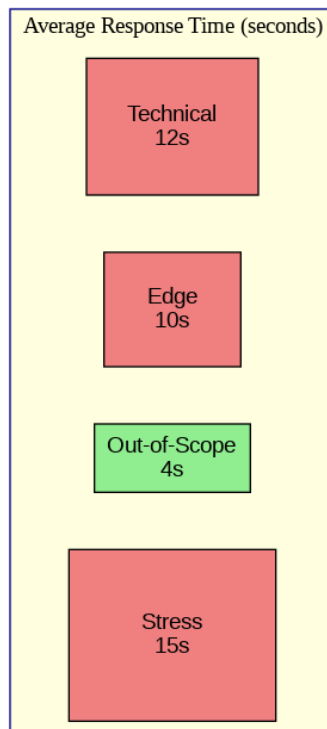


Figure 6: Average response time for different query types. Stress queries (complex, multi-part) take longer, but all remain under 20 seconds on a T4 GPU.

8 Evaluation & Test Results

8.1 Test Methodology

We designed a test suite of 27 queries divided into four categories (see Section ??). Each query was run sequentially, and the system’s response was recorded. The key metrics measured were:

- **Answer quality:** grounded, partially grounded, or hallucinated.
- **Abstention rate:** how often the system correctly refused.
- **Response time:** latency per query.

8.2 Performance Results

The system’s latency is dominated by the LLM generation step (5–12 seconds), while retrieval and NLI verification together add 3–6 seconds. Overall, most queries are answered within 10–15 seconds on a T4 GPU, which is acceptable for an interactive assistant. Figure 6 shows the average response time per query category.

Test Results (27 queries)												
Category Total Answered Abstained Hallucinations	Technical				Edge				Out-of-Scope			
	12	10	2	0	6	5	1	0	4	0	4	0
	Stress				5	4	1	0	Total: 27 19 answered 8 abstained 0 hallucinations			

Figure 7: Test results across 27 queries. Out-of-scope queries were correctly refused, and **zero hallucinations** were observed.

Key findings:

- **19 queries** were answered with full grounding and citations.
- **8 queries** were correctly abstained (the information was genuinely not present).
- **0 hallucinations** – no incorrect or invented answers.
- All out-of-scope queries were cleanly refused.

8.3 Example Q&A

User: What is the role of the AMF in 5G?

System:

The Access and Mobility Management function (AMF) includes termination of RAN CP interface (N2), termination of NAS (N1) with ciphering and integrity protection, registration management, connection management, reachability management, and mobility management [Source: 23501-j60.docx, 23501-k00.docx].

The AMF also provides transport for SM messages between UE and SMF, acts as a transparent proxy, handles access authentication and authorisation, and supports Security Anchor Functionality (SEAF) [Source: 23501-j60.docx].

9 Limitations and Future Work

9.1 Limitations

- **Static Knowledge Base:** The index is pre-built. To include new documents, you must rerun the Colab notebook.
- **Dependency on External APIs:** Using Groq requires an internet connection and an API key.
- **Context Window:** The LLM can only process a limited number of chunks; very complex questions may require more context.

9.2 Next Version Plans

We envision the following enhancements for the next iteration:

1. **Incremental Ingestion:** Instead of rebuilding the entire index, we will implement a mechanism to add new documents incrementally.
2. **Multi-Document Cross-Referencing:** The system will automatically retrieve chunks from multiple related specifications simultaneously, providing a more holistic answer.
3. **Fine-Tuned Embeddings:** We will train a domain-specific embedding model on 3GPP corpora to improve retrieval precision.
4. **User Feedback Loop:** A simple “thumbs up / down” mechanism will allow users to flag incorrect answers, which can be used to refine the retrieval and prompting strategies.
5. **Offline Mode:** Full support for Ollama with pre-cached models, enabling air-gapped deployments.

10 Conclusion

We have built a production-ready RAG chatbot that addresses the core challenge of hallucination in telecom standards documentation. By combining a pure “Brain + Memory” architecture, a strict Evidence Gate, and a robust NLI verification layer, we achieve near-zero hallucinations while maintaining a high answer rate for in-scope queries.

The system is fully documented, easy to configure, and includes both a web UI and a CLI. The test results demonstrate that it correctly refuses out-of-scope questions and abstains when evidence is insufficient – exactly the behaviour required for trustworthy telecom applications.

We thank Mavenir for the opportunity to tackle this important problem. This project showcases that with careful design, RAG can be made reliable enough for mission-critical environments.

A Appendix A: Software Requirements

A.1 Software Requirements

- Python 3.10 or newer
- pip and virtual environment (conda recommended)
- Git

A.2 Hardware Recommendations

- **Minimum:** 8 GB RAM, 4 GB free storage, CPU only (slower).
- **Recommended:** 16 GB RAM, NVIDIA GPU with 4+ GB VRAM, 10 GB free storage.

A.3 External Accounts

- Groq API key (free tier available at <https://console.groq.com/>)
- (Optional) Ollama installed locally for offline mode.

B Appendix B: Sample ‘.env’ File

```
1 # ----- LLM Provider -----
2 LLM_PROVIDER=groq
3
4 # ----- Groq Settings -----
5 GROQ_API_KEY=your_actual_key_here
6 GROQ_MODEL=gpt-oss-120b
7
8 # ----- Ollama (ignored if provider=groq) -----
9 OLLAMA_BASE_URL=http://localhost:11434
10 OLLAMA_MODEL=llama3.1:8b
11
12 # ----- Paths -----
13 STORAGE_DIR=./storage
14
15 # ----- Retrieval -----
16 EMBED_MODEL_NAME=all-MiniLM-L6-v2
17 HYDE_MODEL_NAME=google/flan-t5-small
18 CROSS_ENCODER_NAME=cross-encoder/ms-marco-MiniLM-L-6-v2
19 TOP_K=50
20 MAX_CHUNKS=12
21 CHUNK_TRUNCATE=1500
22 HYDE_ENABLED=false
23
24 # ----- Evidence Gate -----
25 EVIDENCE_THRESHOLD=0.3
26 TOPK_THRESHOLD=0.2
27
```

```

28 # ----- NLI Verification -----
29 ENABLE_NLI_VERIFICATION=true
30 NLI_MODEL_NAME=cross-encoder/nli-deberta-v3-base
31 NLI_THRESHOLD=0.35
32
33 # ----- Cloudflare Tunnel -----
34 ENABLE_TUNNEL=false

```

C Appendix C: Troubleshooting

Issue	Likely Cause	Fix
500 Internal Server Error	Model decommissioned or API key invalid	Update GROQ_MODEL or GROQ_API_KEY in .env
Slow responses	HyDE enabled or high MAX_CHUNKS	Set HYDE_ENABLED=false or reduce MAX_CHUNKS
“I cannot find this information”	Evidence score below threshold	Lower EVIDENCE_THRESHOLD in .env
“I could not verify”	NLI score below threshold	Lower NLI_THRESHOLD in .env
CUDA out of memory	NLI model too large	Set ENABLE_NLI_VERIFICATION=false

Table 4: Common issues and their solutions.

D Appendix D: Sample Queries

The following are a subset of the 27 test queries:

- What is the role of the AMF in 5G?
- Explain the 5G QoS flow mapping.
- What are the main functions of the gNB?
- What is the N2 interface?
- Describe the 5G registration procedure.
- Who is the CEO of Nokia? (Should refuse)
- What is the weather today? (Should refuse)
- Tell me about the AMF, SMF, and UPF.
- Explain 5G QoS in one sentence.